

Role-Based Task Management Portal

A high-contrast, strictly-typed task management system designed for engineering teams. Built with a "Neo-Brutalism" aesthetic, this portal allows for efficient role-based member management and task assignment.

Features

1. Member Management

- Add and view members with details (Name, Role, Email).
- Members are automatically grouped by their roles in the Directory.
- Track active and total tasks assigned to each member.

2. Task Assignment System

- Create tasks with a Title, Description, Assigned Member, and Status.
- **Task Deadlines:** Set optional deadlines during task creation.
- **Kanban Board:** Drag-and-drop tasks between different statuses (Pending, Todo, In Progress, In Review, Done, etc.).
- **Member Task View:** View a list of specific tasks assigned to each team member in the Directory.

3. Dashboard View

- High-level stats on task completion and member activity.
- Team breakdown by role with member names.
- Quick view of recent tasks and tasks assigned per member.

Tech Stack

- **Frontend:** Next.js 14, Tailwind CSS, dnd-kit (Kanban), Lucide React.
- **Backend:** Node.js, Express, TypeScript.
- **Database:** MongoDB (Mongoose).

Setup Instructions

Prerequisites

- Node.js (v18+)
- MongoDB (Running locally or a Cloud URI)

[!WARNING] Important Development Note: Do not run this project using SRM WIFI in a local MongoDB environment. SRM WIFI has known compatibility issues with local database loading in this architecture. Use a standard independent MongoDB instance for stable development.

1. Clone the Repository

```
git clone <repository-url>
cd Taskie
```

2. Backend Setup

1. Navigate to the backend directory:

```
cd backend
```

2. Install dependencies:

```
npm install
```

3. Create a `.env` file in the `backend/` root:

```
PORT=5000  
MONGODB_URI=mongodb://localhost:27017/taskie  
ADMIN_ID=your-admin-id-here
```

4. Start the server (Development):

```
npm run dev
```

3. Frontend Setup

1. Open a new terminal and navigate to the frontend directory:

```
cd frontend
```

2. Install dependencies:

```
npm install
```

3. Create a `.env.local` file in the `frontend/` root:

```
NEXT_PUBLIC_API_URL=http://localhost:5000  
NEXT_PUBLIC_ADMIN_ID=your-admin-id-here  
NEXT_PUBLIC_CURRENT_USER_ID=your-admin-id-here
```

4. Start the development server:

```
npm run dev
```

5. Open <http://localhost:3000> in your browser.

4. Initial Super-User Setup (President)

If you are starting with a fresh database, you must create the first **PRESIDENT** (Super-User) manually:

1. Use Postman or `curl` to send a `POST` request to `http://localhost:5000/api/users`.

2. USE THE FOLLOWING JSON BODY:

```
{
  "name": "Super User",
  "email": "president@taskie.dev",
  "jobTitle": "President",
  "accessLevel": "PRESIDENT"
}
```

3. Copy the `_id` from the response.
 4. Paste this `_id` into your `.env` and `.env.local` files for `ADMIN_ID` and `CURRENT_USER_ID`.
 5. Once this first **PRESIDENT** is set up, they can add more **ADMINS** or even other **PRESIDENTS** directly through the "**Manage Team**" portal in the UI. **ADMINS** can also add other **ADMINS**, but only a **PRESIDENT** has supreme authority over system-wide settings.
-

Deliverables

- **Source Code:** Full project structure provided.
- **Setup Instructions:** Contained in this README.
- **Logic:** Implements all role-based grouping and task assignment requirements.

📅 Project Roadmap

- ☒ **Phase 1:** Role-Based Architecture Design.
- ☒ **Phase 2:** Frontend & Backend Scaffold.
- ☐ **Phase 3:** Core Feature Implementation (Resolve pending TODOs).
- ☐ **Phase 4:** Integration & Full-Stack Polish.